SOFTWARE TESTING

Lecture Notes

From unit tests to complete user journeys

Java 21 | Spring Boot 4 | JUnit 5 | AssertJ | Mockito
Testcontainers | REST Assured | Playwright

A student-friendly, practical teaching edition
Cloud Sphere · one fee rule, followed until a paid order

Learning outcomes

By the end of these notes you should be able to explain why software is tested, choose an appropriate test level, write a readable JUnit test, use assertions and mocks correctly, test a Spring MVC boundary and a real database, check a JSON API, and automate one critical browser journey.

- Distinguish unit, integration, and end-to-end tests by the boundary the test crosses.
- Apply Arrange–Act–Assert, and name a test so a failure can be read aloud.
- Use JUnit 5, AssertJ, and Mockito on ordinary Java, before you start Spring.
- Name the system under test, and decide which collaborators stay real, which are stubbed, and which you verify.
- Use a Spring Boot test slice and Testcontainers when the bug lives on a boundary.
- Assert the JSON fields a client relies on, including the difference between missing, null, and empty.
- Use Playwright for one user journey, and read a timeout as a clue rather than a reason to sleep.
- Treat a coverage percentage as a floor under the evidence, not as a grade for the tests.
- Know which free chapter to open when a question needs a longer argument than these notes.

1. Why we test software

Software testing is a disciplined way to gather evidence that a system behaves as expected. A test sets up a situation, performs an action, and checks an observable result. A useful test fails when the behaviour is wrong and passes when the behaviour is right.

Testing is not a proof that a program is perfect. A finite set of tests cannot cover every input, every environment, and every user. You choose the tests that give strong evidence about the rules that matter and the boundaries where those rules change.

Keep three words separate. The 500 a customer is charged, when the rule required 450, is a failure: you can see it. The defect in the discount line is a fault: it lives in the code. The 450 you wrote down before you ran anything is the oracle: it is how you decide pass or fail. Without an oracle you are only watching the program run.

> **Core idea:** A test is valuable because of what it checks, not because it executes a line of code.

Example: a fee rule

A shop gives 10 percent off when a customer buys five or more items. Four items at 100 each cost 400. Five items cost 500 before the discount and 450 after it, because the method uses integer arithmetic: subtotal times 90, then divided by 100. A negative quantity is refused. Tests belong on both sides of the threshold, and on the refusal.

```
assertThat(FeeCalculator.total(4, 100)).isEqualTo(400);
assertThat(FeeCalculator.total(5, 100)).isEqualTo(450);
```

The first line protects the full-price rule. The second protects the threshold. If someone changes quantity >= 5 into quantity > 5, the second assertion goes red and the first stays green. Six items stay

in the same discount group as five, so one extra case is enough to guard the arithmetic: six at 100 cost 540. You do not need a test for seven, eight, and nine.

2. The three levels of testing

Choose a level by asking which boundary the test must cross to expose the bug. Start with the smallest, fastest test that can reveal it. Add a wider test only where the collaboration itself can fail.

Level	What it checks	Typical tools	Example failure
Unit	One rule or class, in isolation	JUnit 5, AssertJ, Mockito	Wrong discount, or a missing exception
Integration	Components working together across a boundary	Spring slices, MockMvc, Testcontainers	Wrong JSON, bad SQL, a security rule
End to end	A user journey through the running system	REST Assured, Playwright, a real port	The button calls the wrong URL

A common misconception

The annotation does not decide the level. @SpringBootTest with MockMvc starts the application and keeps the request inside the process. That is an integration test. The same annotation with webEnvironment = RANDOM_PORT, plus a real HTTP client, reaches a listening server. Playwright adds the browser and the page. A unit test of the fee needs none of those.

A repository test is justified when the risk is PostgreSQL: a constraint, a type, a migration. A missing redirect on the shop page cannot be found by MockMvc, because MockMvc never loads the frontend.

A practical pyramid

- Write many fast unit tests for rules and for the edges of those rules.
- Write a focused integration test where MVC, persistence, security, or a migration can be wrong on its own.
- Keep end-to-end tests few, and spend them on journeys a customer actually takes. They are slower, and more of the system has to be alive.

3. How to write a readable test

Arrange, act, assert

Each test tells a short story in three parts. Arrange creates the object and prepares the inputs. Act calls one behaviour. Assert compares what came back with the oracle. Keep the three parts visually separate. After a week you will see them without the comments, and a classmate should still be able to point at each one.

```
@Test
void add_twoAndThree_returnsFive() {
    // Arrange
    Calculator calculator = new Calculator();

    // Act
    int sum = calculator.add(2, 3);

    // Assert
    assertThat(sum).isEqualTo(5);
}
```

Name the case so a failure can be read aloud

Use three parts, separated by underscores: subject, condition, outcome. Use camelCase inside each part. The subject is the method or the route. The condition is the input that matters. The outcome is what the oracle demands.

```
divide_byZero_throwsIllegalArgument()
total_fiveItems_appliesTenPercent()
count_blankOrNull_returnsZero()
```

`total_fourItems_chargesFullPrice` reads as: total, given four items, charges full price. A name that joins two outcomes with “and” is two tests.
`total_fourItems_chargesFullPrice_and_fiveItems_appliesTenPercent` should be split, because a red bar would not say which oracle broke. Two inputs may stay in one method only when they share one outcome, and even then blank and null hide different faults, so prefer two methods when you are hunting those faults.

Assertions

AssertJ reads the actual value first and the expected value second, which matches the way you talk: the sum is 5, the thrown exception is this type, the message is this sentence. AssertJ comes with spring-boot-starter-test.

```
assertThat(sum).isEqualTo(5);
assertThat(found).isNull();
assertThatThrownBy(() -> Calculator.divide(10, 0))
        .isInstanceOf(IllegalArgumentException.class)
        .hasMessage("divisor must not be zero");
```

4. Unit testing: test the rule, not the framework

A unit test exercises a small piece of behaviour without starting Spring, opening a database, or launching a browser. It should be fast, deterministic, and cheap to run while you are still typing.

Example A: a calculator

Addition has an oracle. Division by zero has a different oracle, an exception, so it is a different test. Division of 5 by 2 returns 2, because the method uses integer division. That fact deserves its own test, so nobody repairs it into a float by accident.

```
public final class Calculator {
    public int add(int a, int b) { return a + b; }
    public int divide(int a, int b) {
        if (b == 0) {
            throw new IllegalArgumentException(
                    "divisor must not be zero");
        }
        return a / b;
    }
}
```

```
@Test
void divide_byZero_throwsIllegalArgument() {
    Calculator calculator = new Calculator();
    assertThatThrownBy(() -> calculator.divide(10, 0))
            .isInstanceOf(IllegalArgumentException.class)
            .hasMessage("divisor must not be zero");
}

@Test
void divide fiveByTwo returnsTwo() {
    Calculator calculator = new Calculator();
    assertThat(calculator.divide(5, 2)).isEqualTo(2);
}
```

When the interesting result is the exception, the act and the assert are the same expression. Say so in a comment. When they are separate, as they are for the fee, keep them separate.

Example B: null and blank are different faults

A word counter returns 0 for null and for blank text, and it must not count the gaps between words as extra words. Null and a string of spaces both return 0, and they are still two cases. A method that calls isEmpty instead of isBlank throws on null and counts the gaps as words. One case cannot catch both mistakes.

```
public final class WordCounter {
    public int count(String text) {
        if (text == null || text.isBlank()) {
            return 0;
        }
        return text.trim().split("\\s+").length;
    }
}
```

The cases worth writing are null, an empty string, spaces only, one word, and two words with a repeated gap. Splitting on a single space fails the repeated-gap case. The call in the box is the whitespace regex, written the way a Java string requires: two backslashes in the source, so the pattern sees whitespace.

Example C: the fee threshold

The method is public and static because it has no fields and no repository. The test calls it by name. You do not mock a pure function you own.

```
public final class FeeCalculator {
    public static long total(int quantity, long unitPrice) {
        if (quantity < 0 || unitPrice < 0) {
            throw new IllegalArgumentException(
                    "quantity and unitPrice must be non-negative");
        }
        long subtotal = quantity * unitPrice;
        return quantity >= 5 ? subtotal * 90 / 100 : subtotal;
    }
}
```

Input	Partition	Oracle
quantity −1, price 100	Refused	IllegalArgumentException
quantity 0, price 100	Empty, and legal	0
quantity 4, price 100	Last full price	400
quantity 5, price 100	First discounted	450
quantity 6, price 100	Same discount group	540

For a threshold, test just below, exactly on, and just above the line. Zero is a legal empty order. Negative is the refusal. A test that expects 450.0 has changed the contract into floating point.

Static methods, private helpers, and the clock

- Call a public static function directly. Mockito can rewrite a static method you do not own. This fee method is ours.
- Test a private helper through the public method whose behaviour depends on it. A test that uses reflection to call the helper goes red when someone renames the helper, even if the count a customer sees is still correct.
- Pass time in as a value. Clock.fixed gives you one instant, so a due time one second ahead is not overdue and a due time one second behind is overdue. Instant.now() moves while the test runs. Thread.sleep waits for the wall clock and makes the test slow.

```
private final Clock clock;

public Deadline(Clock clock) { this.clock = clock; }

public boolean isOverdue(Instant due) {
    return clock.instant().isAfter(due);
}
```

When work finishes later, on another thread, poll for the condition with Awaitility and a deadline. Sleeping for a fixed duration can return before the work has finished.

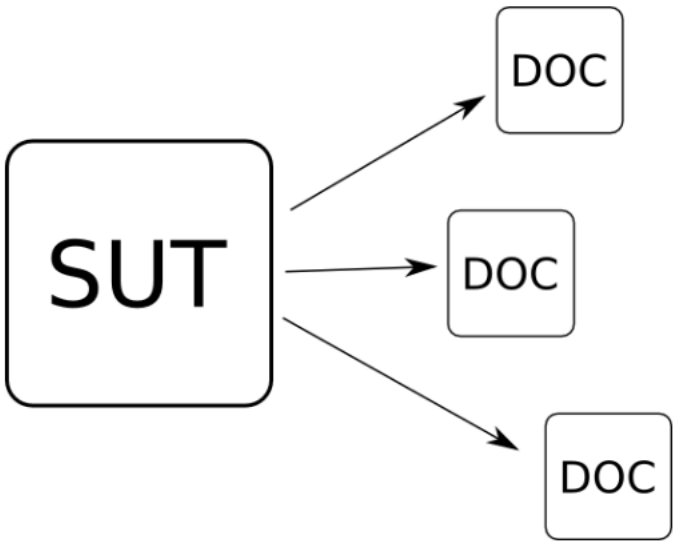
5. The system under test and its collaborators

Before you choose a mock, name two things. The system under test, the SUT, is the behaviour this test is allowed to go red for. A collaborator is something that behaviour calls in order to do its work. The SUT stays real. A collaborator may stay real, or you may replace it.

By **SUT**, or **System Under Test**, we understand an entity that is being tested. Depending on types of tests SUT might be of very different granularity - from single class to the whole application.

BTW. SUT acronym is used in few variants: first word is expanded as "system", "structure" or "subject".

A **DOC**, or **Depended On Component**, is a collaborator, any entity that is required by SUT to fulfill its duties. Usually DOC is of same granularity as SUT, e.g. if SUT is a class, then it uses other classes, if it is a module, then it depends on other modules, etc.



What the system under test is

The SUT can be a method, a class, a Spring service, a controller, or one business rule. In CalculatorTest the SUT is Calculator. You write new Calculator(). Addition has an oracle, and division by zero has a different oracle. There is no repository, so there is nothing to replace.

In AdminCatalogServiceTest the SUT is createCategory. The test may go red when the name is not trimmed, when a new name is not saved, or when a duplicate is saved anyway. It must not go red because PostgreSQL is stopped. That failure belongs to a different test.

The same class is not the SUT in every test. In a controller slice the controller is the SUT and AdminCatalogService is a collaborator. In ProductRepositoryTest the repository is the SUT, and the service is not in the room. You choose the SUT by asking which fault this test should expose.

What a collaborator is

A collaborator is a dependency the SUT uses. createCategory asks CategoryRepository whether the name exists, and it asks that repository to save. CountService asks WordCounter for a count, and it asks CountStore to save when the count is not zero. A transfer service has the same shape: the service is the SUT, and the account repository and the notification sender are collaborators.

A collaborator does not have to be a mock. The word only means the SUT depends on it. You then choose a real object, a stub, a mock, or a fake.

Piece	Role in that test	Why
AdminCatalogService	SUT of the service test	Holds the rule: trim, save, or refuse.
CategoryRepository	Collaborator	Looks up a name and saves a row.
Category	Value, not a collaborator	The row the service builds. Keep it real.
Calculator	SUT of CalculatorTest	No collaborators. Do not mock add.
WordCounter	Collaborator of CountService	A pure function. It can stay real.
CountStore	Collaborator of CountService	The save. Mock it in the service test.

> **Core idea:** A collaborator is a dependency. A mock is one way to supply that dependency in a unit test. ProductRepositoryTest uses the real repository, because there the repository is the SUT.

Why you separate them

Suppose the case is a duplicate category name. You want evidence about the rule, not about the lab network. If the test opens PostgreSQL, a stopped Docker daemon fails the test while createCategory is still correct. Replace the repository with a stand-in, and the red bar means the rule.

Three things improve when you separate the SUT from the collaborators you do not intend to test:

- Isolation. A failure points at the component this test was written for.
- Control. You decide what findByNameIgnoreCase returns: empty, or a category that already exists.
- Verification. When the requirement says a duplicate must not be stored, you can check that save did not run.

Isolation is not a reason to leave the collaborator untested. Each collaborator that has its own rule gets its own test, with itself as the SUT. The way the pieces work together is an integration test. AdminControllerTests starts the service and PostgreSQL together, and it mocks neither.

A unit test, with the roles named

Mockito builds the stand-ins and passes them into the real service. In the project the field is called service. That field is the SUT. The two @Mock fields are collaborators. @Mock here is a Mockito annotation on a plain JUnit test. It is not @MockitoBean, which replaces a bean inside a Spring test. That annotation belongs in section 7.

```
@ExtendWith(MockitoExtension.class)
class AdminCatalogServiceTest {

    @Mock ProductRepository products;
    @Mock CategoryRepository categories;
    @InjectMocks AdminCatalogService service;

    @Test
    void createCategory existingName throwsConflictAndDoesNotSave() {
        // Arrange: the collaborator already has this name
        when(categories.findByNameIgnoreCase("Phones"))
            .thenReturn(Optional.of(new Category("Phones")));

        // Act
        ApiException ex = assertThrows(ApiException.class,
                () -> service.createCategory(" Phones "));

        // Assert
        assertEquals(HttpStatus.CONFLICT, ex.status());
        verify(categories, never()).save(any());
    }
}
```

Walk the method in the order the test runs.

- @Mock builds a stand-in for a collaborator. It does not open a database.
- @InjectMocks builds the SUT and passes those stand-ins into the constructor. The production class keeps an ordinary constructor.
- when(...).thenReturn(...) is Arrange. It scripts the collaborator before the call. Here the lookup finds Phones.
- The call on service is the act. The exception is the result you can see. The project also checks the message: a category with that name already exists.
- verify(..., never()) is the other assert. The contract includes a conversation that must not happen: save.

ProductRepository is also a constructor argument, so the test mocks it too, even though createCategory does not call it. @InjectMocks has to be able to call the real constructor. The other method in the same class, createCategory_newName_savesTheTrimmedName, stubs an empty lookup and verifies that save received the trimmed name Phones. Stubbing save does not prove a unique key. It proves only that the service asked.

Stub, mock, fake, and the real object

These are all test doubles, except the last row. The difference is what you use the double for, not which library constructed it. A Mockito object can be a stub, a mock, or both. when makes it a stub. verify makes it a mock.

Kind	You use it to	In this course
Stub	Return a prepared answer	findByNameIgnoreCase returns a category, or returns empty.
Mock	Check that a call did or did not happen	verify that save ran once, or never.
Fake	Run a small real implementation	CalculationRepository stores rows in a map, so the size can grow and shrink.
Real object	Run the production code	new Calculator(), and the real ProductRepository on PostgreSQL.

Stub a collaborator when the SUT needs an answer in order to reach the case. Verify an interaction when that interaction is part of the requirement. Do not verify every call only because it happened. A test that checks the sequence of private steps goes red when someone renames a helper, even if the category a customer sees is still right.

WordCounter is a small pure function. The CountService exercise asks you to mock it, so a canned 2 proves that the service saved whatever count returned, and a canned 0 proves that a blank string does not save. You could also leave WordCounter real, and let the service test use a true word count. Mocking it does not prove the count. A test of WordCounter, with no mocks, does that. Calculator.add must stay real. A mock of add would replace the rule the test was written to catch.

When a collaborator should be mocked

Situation	Typical choice
The service rule, and the repository would open PostgreSQL	Mock the repository. AdminCatalogServiceTest.
The word count inside CountService	Either a stubbed WordCounter, or the real one.
Calculator.add	Real object. A mock would replace the rule.
Whether the SQL and the unique key are right	Do not mock. ProductRepositoryTest, on PostgreSQL.
Controller, service, and database together	Integration test. No mock of the repository.
One admin journey on a real port	End-to-end. The journey is the SUT. The services stay real.

The same classes, three different SUTs

The level of the test is the boundary it crosses. The SUT is whatever sits inside that boundary.

- Unit test. The SUT is AdminCatalogService. CategoryRepository is mocked. Spring does not start.
- Repository slice. The SUT is ProductRepository. PostgreSQL is real. The service is absent.

- End-to-end. The SUT is the admin journey: sign in, publish, set stock, ship, deliver. AdminJourneyTests uses a real port. A browser journey is a wider SUT, and these notes have not automated that page yet.

Section 14 asks you to name the SUT in createCategory, and to say whether every collaborator must be mocked.

> **Remember:** The SUT is what this test is trying to catch. Collaborators are what it depends on. Control the dependencies the case needs, and check the behaviour the requirement promised.

6. Assertions, stubs, and verification

Once the SUT is named, a stand-in replaces a collaborator, such as a repository or a payment gateway, so the class you are testing can stay real. A stub is configured to return a canned result. A mock is the same kind of stand-in when you also check that it was called. The word people use in conversation is often just “mock”. The check you write tells you which one you built.

The result, and the conversation

An assertion answers: what did the method produce? A Mockito verify answers: did it talk to the collaborator correctly? They are different promises.

```
int n = service.countAndStore("hello world");
assertThat(n).isEqualTo(2);
verify(store).save("hello world", 2);
```

The assertion alone stays green if the service returns 2 and forgets to save. The verify alone stays green if it saves 2 and returns 99. When both are part of the contract, check both. verify(store, never()).save(...) is the check that an invalid amount must not be stored.

A service test

`@ExtendWith(MockitoExtension.class)` creates the stand-ins. `@Mock` is the stand-in. `@InjectMocks` constructs the real service and passes the stand-ins into its constructor, so the production code keeps a normal constructor and the test does not call new and forget a dependency.
`when(...).thenReturn(...)` is Arrange. `verify` belongs after the act.

```
@ExtendWith(MockitoExtension.class)
class PaymentServiceTest {
    @Mock PaymentRepository payments;
    @InjectMocks PaymentService service;

    @Test
    void charge positiveAmount savesThePayment() {
        when(payments.save(any(Payment.class)))
                .thenReturn(new Payment(1L, 500));

        Payment result = service.charge(500);

        assertThat(result.id()).isEqualTo(1L);
        assertThat(result.amount()).isEqualTo(500);
        verify(payments).save(any(Payment.class));
    }
}
```

`@Mock` is not `@MockitoBean`. The first lives in a test with no Spring context. The second replaces a bean inside a Spring test. Mixing them up is a common reason a controller test never sees the stub.

> **Limit:** A mocked repository does not prove SQL, a mapping, a migration, a unique key, or a transaction. Those need a real database, which is the next two sections.

7. Integration testing with Spring Boot

An integration test crosses a boundary between components. Spring Boot can start a slice of the application, so you do not boot the whole shop to check one controller mapping.

Tool	Useful for	It does not, by itself, prove
@WebMvcTest and MockMvc	The route, the parameters, validation, and the JSON	The real fee arithmetic, or the database
@DataJpaTest	Entity mappings and repository queries	A Flyway migration, or the shop page
@SpringBootTest and MockMvc	The full context, with HTTP still in-process	A real socket, or a browser
@SpringBootTest with RANDOM_PORT	A running server reached over HTTP	That the page looks right

A controller slice

The service is replaced, so this test cannot prove the fee. The stub returns 9 for a quote of 2 and 3, which is not the real product. If the JSON comes back as 9, the controller passed the parameters through and wrote the field. The arithmetic stays in FeeCalculatorTest.

```
@WebMvcTest(FeeController.class)
class FeeControllerTest {
    @Autowired MockMvc mvc;
    @MockitoBean FeeService fees;

    @Test
    void total_validParameters_returnsJsonTotal() throws Exception {
        when(fees.quote(2, 3)).thenReturn(9L);

        mvc.perform(get("/api/fees")
                .param("quantity", "2")
                .param("unitPrice", "3"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.total").value(9));
    }
}
```

This project is Spring Boot 4. Match the imports to that release, because a snippet from an older tutorial will not compile.

```
org.springframework.boot.webmvc.test.autoconfigure.WebMvcTest
org.springframework.boot.data.jpa.test.autoconfigure.DataJpaTest
org.springframework.test.context.bean.override.mockito.MockitoBean
```

Older tutorials say `@MockBean`. On this release the replacement bean is `@MockitoBean`. `@Mock` remains the right tool when the test has no Spring context at all.

8. A real database

A repository stand-in never opens a connection. It cannot reveal invalid SQL, a PostgreSQL constraint, a broken migration, or a Java type that does not match the column. Use the same database family you will ship when those are the risks.

What Testcontainers is doing

Testcontainers starts PostgreSQL or Redis in Docker for the test, points Spring at that container, and stops the container when the class is finished. You do not install a database on your laptop for the test, and you do not share one with the next person in the lab.

- Use a random published port. Port 5432 on your machine is often already taken, and it is not the port inside the container.
- Keep a container field static when the whole class should share one database. A new container per method is slow and usually accidental.
- A skipped test is not a passing assertion. If Docker is stopped, @Testcontainers(disabledWithoutDocker = true) skips the class. Read the summary and check that the tests ran.
- In Spring Boot 4, @ServiceConnection on the container can supply the JDBC URL. The image name for Redis must be redis, or the connection will not bind.

A repository example

```
@DataJpaTest
@AutoConfigureTestDatabase(replace = Replace.NONE)
class InventoryRepositoryTest {
    @Autowired InventoryRepository inventory;

    @Test
    void save newSku canBeLoaded() {
        inventory.saveAndFlush(new StockItem("USB-C-01", 12));
        StockItem found = inventory.findBySku("USB-C-01").orElseThrow();
        assertThat(found.quantity()).isEqualTo(12);
    }
}
```

saveAndFlush sends the SQL immediately, which is what you want when the oracle is a unique-key failure. A JPA slice usually rolls the test transaction back after each method, so the next method starts clean.

@DataJpaTest does not run Flyway. It builds tables from the entities, with ddl-auto set to create-drop, and replace = NONE tells Spring to keep the Testcontainers database instead of swapping in an embedded one. That is the right test for a mapping and a query. It is the wrong test for a migration. If the question is whether V1 applied, start the full application context and let Flyway run. Cleaning that database in a test requires spring.flyway.clean-disabled=false, and only then is flyway.clean() followed by migrate() meaningful.

One test must not feed the next

A full @SpringBootTest does not roll back every write. Isolate rows with a key that belongs to the test, or delete them afterwards. Delete child rows before parent rows when a foreign key requires that order.

@Transactional on a @SpringBootTest method is the wrong tool when the behaviour you care about is a write that must survive an exception. The rollback hides that write. It is also the wrong tool beside RANDOM_PORT: the request is handled on another thread, and it commits, while the test thread’s transaction never sees those rows.

An ApplicationRunner that seeds the catalogue runs when the context starts. It does not run again before the next method. If a test deletes the seed, later methods in that context will not get it back.

```
await().atMost(Duration.ofSeconds(2))
    .untilAsserted(() ->
        assertThat(inventory.reserved(sku)).isZero());
```

9. Testing a JSON API

JSON is the contract with the client. Check the fields, the types, and the size of a collection. Do not compare one giant string: property order and insignificant spaces will fail a correct response. JSONPath points at the part you care about.

One loan, written down first

Decide the row before you ask for it. Loan 101 belongs to Jean Paul, the principal is 1000000, the loan is eligible, and there are two installments of 85000. Put that row in the test database, or stub the service in a slice. Do not depend on a developer's database happening to contain loan 101.

```
{
  "id": 101,
  "customerName": "Jean Paul",
  "principal": 1000000,
  "status": "ACTIVE",
  "eligible": true,
  "installments": [
    { "number": 1, "amount": 85000 },
    { "number": 2, "amount": 85000 }
  ]
}
```

Requirement	JSONPath	Expected
Loan id	$.id	101
Customer	$.customerName	Jean Paul
Eligible	$.eligible	true
First amount	$.installments[0].amount	85000
How many	$.installments.length()	2

```
mvc.perform(get("/api/loans/101"))
    .andExpect(status().isOk())
    .andExpect(jsonPath("$.customerName").value("Jean Paul"))
    .andExpect(jsonPath("$.installments.length()").value(2))
    .andExpect(jsonPath("$.installments[0].amount").value(85000));
```

Missing, null, and empty

- A missing property is not a property whose value is null.
- Null is not an empty string, and an empty array is not an omitted array.
- Assert the fields the client uses. Leave incidental fields alone.

On the Spring 7 jsonPath used here, exists() means a non-null value, so a stored null does not exist. hasJsonPath() means the key is present, and it allows null. doesNotHaveJsonPath() is the matcher for a key that was omitted. An empty array still exists. MockMvc writes these checks with a leading $. REST Assured's body() uses Groovy GPath, so the same idea is installments.size() with no dollar sign. Spring Boot 4 does not manage REST Assured; this course pins io.rest-assured:rest-assured 5.5.7.

A controller slice does not load @ControllerAdvice unless you import it. A 404 body produced by the advice belongs on a test that starts the wider context, or on a slice that imports the advice on purpose.

10. A browser journey with Playwright

An end-to-end test walks a journey the way a person does. The page, the navigation, the validation message, the proxy, the API, and the saved row can all be in that path. Fee arithmetic can be green, and the API can be green, while the Pay button still calls /api/order and the server exposes /api/orders. A click on Pay is what reveals that.

Prefer locators a person could also use: the label, the role, the accessible name. getByLabel finds a field only when the label is associated with it. A label that is visual but not wired up will time out, and the fix is the label, not a longer timeout. A test id is reasonable for an element the page deliberately exposes, such as the order status.

```
page.getByLabel("Email").fill("ada@shop.test");
page.getByLabel("Password").fill("Secret1a");
page.getByRole(AriaRole.BUTTON,
        new Page.GetByRoleOptions().setName("Sign in")).click();
PlaywrightAssertions.assertThat(
        page.getByTestId("order-status")).hasText("PAID");
```

Check the screen and the stored row. The page can say PAID while the database has nothing, and the database can have a paid order the page never shows.

- A valid customer signs in, pays, and sees PAID.
- An anonymous visitor who opens a protected page is sent to sign in.
- A blank email shows the validation message.
- A declined payment stays declined. It must not become PAID, and a timeout from the payment stub must not become PAID either.
- The test's own rows are the only rows it reads, so a second run does not trip over the first.

Playwright already waits until a locator can be clicked, and it retries assertions until they pass or time out. Thread.sleep does not repair a wrong locator. Use a random port and start the page yourself in the setup, so the test does not depend on a server a classmate left running. This course uses Playwright Java 1.63.0. Keep the fee boundaries in the unit tests, where a wrong 450 is obvious. Spend the browser on one happy payment and the few journeys that can only fail in the browser.

> **Scope:** A browser test confirms a small number of important journeys. It is a poor place to rediscover every threshold from section 4.

11. What a test cannot prove

Each technique has a boundary. Strong testing is knowing what evidence you have and what you still do not have.

A test cannot prove	Why	Use this as well
SQL, by mocking a repository	No SQL runs	A repository test on PostgreSQL
Arithmetic, by stubbing the service	The stub invented the number	A unit test of FeeCalculator
The shop page, with MockMvc	The browser is not loaded	One Playwright journey

A test cannot prove	Why	Use this as well
A real card was charged, when the gateway is a stub	The provider is simulated	A sandbox check, outside this suite
The layout, from a JSON assertion	JSON has no pixels	The page, or a person looking
Correctness, because coverage is high	Coverage counts execution	Oracles, boundaries, and a check that would go red
The database, when the container test was skipped	The assertions never ran	Docker up, then read the test count
Many customers at once	These tests are one user	The load and stress notes

The JaCoCo line gate in this project is 85 percent, and it is a floor. `./mvnw verify` fails the build under that gate. A test can execute a line, assert nothing meaningful, and still raise the number. Branch coverage tells you which decisions ran. Neither number chooses the oracle. That choice was section 1.

12. Running the tests, and reading a red bar

While you are writing, run the smallest group that can go red. Before you merge, run the wider suite. The groups in this project are tags: unit, integration, and e2e.

```
./mvnw -Dgroups=unit test
./mvnw -Dgroups=integration test
./mvnw -Dgroups=e2e test
./mvnw verify
```

`./mvnw verify` is a Maven phase. It runs the tests and the coverage gate. Mockito's verify is a different word: it checks one call inside one test. If a classmate says "verify failed", ask which of the two they mean.

Surefire, as configured here, runs classes named *Test, *Tests, and *TestCase. A class named *IT is compiled and then skipped. There is no Failsafe plugin in this build, so do not rename a test to CalculatorIT and expect it to run. An empty e2e group needs the Surefire flag that allows a group to contain no tests, or the build looks like a failure when the group is simply empty.

Unit tests do not need Docker. Integration tests do. In GitLab CI the unit job has no Docker service. The integration job and the gate use a Docker daemon. On your laptop the daemon socket is the one Docker Desktop opens. A green pipeline whose integration tests were skipped is not evidence about PostgreSQL.

When the bar is red

- A compile error: check the import against section 7, and check that the snippet matches Boot 4.
- A Docker error: start Docker, and confirm the test is using the container's mapped port.
- A missing table: decide whether this slice builds tables from entities, or whether Flyway should have run.
- A leftover row: look at cleanup, and at whether @Transactional hid a commit or rolled back a write you needed.
- Zero tests, or skipped tests: read the count. A green summary can be a suite that never asserted anything.
- A Playwright timeout: check the URL, the accessible name, whether the label is associated, and whether the page and the API are up, before you raise the timeout.

13. A guided lab

Move from one rule to one journey. Commit each stage on its own, and write one sentence on what new evidence that stage added. The scenario is a small checkout. Quantities below zero are rejected. Five or more items earn 10 percent off, in integer arithmetic. A successful payment creates one PAID order. A declined payment must not.

1. Implement FeeCalculator.total. Test four items, five items, six items, zero, and a negative quantity.
2. Implement a PaymentService that calls a PaymentRepository. With Mockito, a valid charge saves once, and an invalid amount does not save.
3. Add a controller and a @WebMvcTest. Assert the status and the JSON field. Stub the service with a number that is not the real product, so you can see what the slice proves.
4. Add a repository test on PostgreSQL with Testcontainers. Check that a saved row can be loaded, and that a duplicate key fails.
5. Add an HTTP test on a random port. Check the response and the row that was stored.
6. Add one Playwright journey: sign in, pay, and check that the status on the page and the status in the database agree.
7. Run the four commands from section 12. Record which tests are fast, which need Docker, and which need a browser.

Questions for the lab report

- Which defect would each stage catch? Give one concrete example.
- Which test uses a stand-in, and what does that stand-in deliberately not prove?
- Which object was the SUT, and which objects were collaborators? Which collaborator did you leave real, and why?
- Why test both four items and five items?
- How do you keep one test’s rows out of the next test?
- Which failure is the easiest to diagnose, and which is the hardest? Why?

14. Review questions

Answer these before you read the key. Where a question uses the fee, the rule is the one from section 1: below five items the customer pays the full subtotal, from five items the customer pays 90 percent in integer arithmetic, and a negative quantity is refused.

1. Which test level is the right one for “five items trigger 10 percent off”?
2. Does @SpringBootTest with MockMvc count as an end-to-end test? Explain.
3. What is the difference between assertThat(result) and verify(repository).save(...)?
4. Why can a mocked repository not prove a PostgreSQL unique constraint?
5. Why point JSONPath at individual fields, instead of comparing one large JSON string?
6. Why avoid Thread.sleep when the work finishes later?
7. What does a Playwright test add beyond a REST Assured call?
8. Why is a skipped Testcontainers test not evidence that the database is correct?
9. What are the three stages of Arrange–Act–Assert?
10. Why is a high line-coverage percentage not enough?

11. In a test of createCategory, which object is the SUT: AdminCatalogService or CategoryRepository?
12. Must every collaborator be mocked in a unit test?
13. Calculator.add has no collaborators. What should the test construct, and what should it not mock?

Answer key

1. A unit test. The discount is a rule. It needs no Spring and no database. Four items expect 400 and five items expect 450.
2. No. MockMvc dispatches inside the process. A real port, or a browser, crosses a wider boundary.
3. assertThat checks an observed value. verify checks that a collaborator was called. A returned 2 can hide a missing save, and a save can hide a returned 99.
4. The stand-in does not execute SQL and does not apply constraints. Load the row on PostgreSQL and expect the duplicate key to fail.
5. Field checks survive property order and insignificant spaces, and they say which part of the contract broke. Remember that a stored null and a missing key are different matchers.
6. Sleep follows the wall clock, wastes time, and can return before the work is done. Freeze time with Clock.fixed, or poll with Awaitility and a deadline.
7. The page, the click, and the wiring from the button to the API. REST Assured never loads the frontend, so it cannot see a Pay button aimed at the wrong path.
8. The assertions did not run. Start Docker and read the test count. A skip is not a pass.
9. Arrange prepares the input and any collaborator. Act calls one behaviour. Assert compares the result with the oracle you chose in advance.
10. Coverage counts the lines that ran. It does not know whether an assertion would catch a wrong result. The gate in ./mvnw verify is a floor under that evidence.
11. AdminCatalogService. CategoryRepository is a collaborator. In ProductRepositoryTest the repository is the SUT instead, and the service is not loaded.
12. No. Mock a collaborator when its real behaviour would open a database or a network, or when the case needs a canned answer. Leave a pure function real when you want its true result. WordCounter may stay real. Calculator.add must stay real, or the test no longer checks addition.
13. new Calculator(). A mock of add replaces the rule the test was written to catch.

15. Glossary

Term	Meaning in these notes
Failure	The wrong result you can observe, such as a charge of 500 where the rule required 450.
Fault	The defect in the code or the configuration that produced the failure.
Oracle	The expected result, chosen before the test runs.
Assertion	A check that an observed result matches the oracle.
SUT	The system under test: the behaviour this test is allowed to go red for. It stays real.
Collaborator	A dependency the SUT calls. It may be real, stubbed, mocked, or a fake. It is not automatically a mock.
Stub	A stand-in configured to return a predetermined result.
Mock	A stand-in you also verify: did the call happen, with these arguments?

Term	Meaning in these notes
Fake	A small working stand-in, such as CalculationRepository storing rows in a map.
Test slice	A Spring test that loads one part of the application, such as MVC or JPA.
MockMvc	Exercises Spring MVC inside the process, with no listening port.
Testcontainers	Starts a real service, such as PostgreSQL, in a disposable container.
JSONPath	Selects a field or an array element in a JSON document. MockMvc writes it with a leading $.
End to end	A journey through the running system, often including the browser.
Coverage	A count of code that executed. Not a measure of whether the oracles are any good.
Boundary	An input where the rule changes. Here, quantity 4 and quantity 5.
Awaitility	Polls until a condition becomes true, or until a deadline.
Idempotency key	An identifier that lets a retried Pay create one order, not two.

The versions in this course are Java 21, Spring Boot 4, JUnit 5, AssertJ, Mockito, Testcontainers, Awaitility, REST Assured 5.5.7, and Playwright Java 1.63.0. Imports move between Boot releases. When a snippet from the internet does not compile, trust the imports in section 7 and the coordinates in the project.

16. Books you can read online

These notes are the path through the fee rule and this codebase. The books below are where you take a question when you want the longer argument. Each one is published free for students by its authors or by its standards body. Read the chapter that matches the section you are in. You do not need to finish a book before the next exercise.

When you are in	Read	Where it is free
Sections 1 and 2	SWEBOK Guide v4, Chapter 5. Then Pezzè and Young, chapters 2, 3, and 10.	computer.org/swebok and the authors' page
Sections 3, 4, and 12	Software Engineering at Google, chapters 11 and 12.	abseil.io/resources/swe-book
Sections 5 and 6	Google, chapter 13, and the pages for Test Double, Stub, and Mock Object.	abseil.io and xunitpatterns.com
Sections 7 to 10	Google, chapter 14, and Pezzè and Young, chapters 21 and 22.	abseil.io and the authors' page
The words an employer uses	ISTQB CTFL syllabus v4.0.1, sections 1 and 4.	istqb.org, CTFL page

SWEBOK v4.0a is the IEEE Computer Society's guide. Digital copies are free for personal and academic use. Chapter 5 names faults, failures, the oracle problem, test levels, and the limits of testing. It is a short map, and it will make the words in section 1 feel standard. Download it from computer.org/swebok.

Mauro Pezzè and Michal Young, Software Testing and Analysis (Wiley, 2008), is the book for the technique behind the fee table. Chapter 10 derives equivalence classes and boundary values from a specification. Chapters 2 and 3 set the vocabulary. Chapter 12 is structural testing, which sits underneath a coverage number. Chapters 21 and 22 are integration, system, and regression testing. The authors give the electronic copy away. Request it from Michal Young's book page. The slides for each chapter are on the same page.

Software Engineering at Google, curated by Titus Winters, Tom Manshreck, and Hyrum Wright, is free as HTML under a Creative Commons licence. Start at the testing overview. Chapter 12 is what a unit test is for. Chapter 13 is the one to read before you call every stand-in a mock. Chapter 14 is the one to read before you add another browser journey. The front of the book is abseil.io/resources/swe-book.

Gerard Meszaros collected the patterns behind sections 5 and 6 at xunitpatterns.com. Test Double, Stub, Mock Object, and state verification versus behaviour verification are the pages to open when a verify is red and you are no longer sure which stand-in you built. The catalogue is free. The full narrative is his book xUnit Test Patterns, which you would buy.

The ISTQB Certified Tester Foundation Level syllabus, v4.0.1, is the glossary a large part of the industry uses. Sections 1 and 4 line up with the opening of these notes: why we test, the difference between testing and debugging, equivalence partitioning, and boundary values. The syllabus is free from the CTFL certification page. Confirm the file says v4.0.1. The exam is optional and is not part of this course.

> **A path:** After section 1, read SWEBOK Chapter 5 through the oracle problem. After section 4, read Pezzè and Young Chapter 10 with your fee table beside you. After section 5, read Google's chapter 13 and the Test Double page. After section 12, read Google's chapter 12: a unit test that is hard to maintain is a design problem.

Two books are worth owning later, and they are not free, so they are not required. Glenford Myers' The Art of Software Testing is the classic on the psychology of looking for failures. Kent Beck's Test-Driven Development: By Example is the classic on writing the test before the production code. If you try one chapter of Beck after section 4, write the next fee test before you change the method.